MATH 3190 Homework 4

Due February 19, 2026
Focus: Notes 6
Total Points: 90 + 10 (Self-Critique Comment and Uploading to GitHub) = 100

Note that your homework should be completed in Quarto. You can just add your answers and code to this document in the appropriate part. The file should be rendered as an html document or pdf document (html is quicker to render and you will need an installation of LaTeX to render as a pdf). You will "turn in" this homework by uploading the files, the .qmd and output file, to your GitHub `Math_3190_Assignment` repository in the `Homework/Homework_4` directory. Make sure all supporting files (if there are any) are in this directory as well. You should use `git` and `Unix` commands in a terminal for this (i.e. don't manually upload files to GitHub). In addition, you will submit just your html or pdf file on Canvas to unlock the solutions.

Don't forget to also put a self-critique comment on Canvas indicating what you did correctly and incorrectly and include any questions you may have about the assignment after submitting the assignment and thoroughly looking through the solutions.

Some of the parts in this homework, like 1a, 2b, 2c, 3a, and 3b require writing down some math-heavy expressions. You may either type it up using LaTeX-style formatting in Quarto (preferred), or you can write it by hand (neatly) and include pictures or scans of your work in your Quarto document.

Problem 1 (33 points)

Suppose we want to find the value of x and the objection function value for finding the global maximum and minimum of the function $f(x) = \ln(1 + x^2)(1 + x)e^{-x^2}$. We will implement gradient descent and Newton's method for finding these extrema.

Part a (5 points)

Find $f'(x)$ for this function. Do this by hand as a nice derivative refresher. You don't need to include all of your work, though. Feel free to check your answer using something like Wolfram|Alpha. Write down the derivative here.

Part b (4 points)

Below is the code of a function that implements gradient descent using a backtracking line search each iteration to find γ_k: the step size. This function takes eight inputs:

- The starting value (or vector)
- The function to optimize
- The function's derivative (or gradient)
- A logical indicating if we want to find a max with a default of `FALSE`.
- The maximum number of iterations with a default of 200
- The initial step size with a default of 1
- The line search beta parameter with a default of 0.5.
- The stopping tolerance with a default of `1e-10`.

The output of this function is a list with the x value (or vector) that optimizes the function, the function value at that point, and the number of iterations it took to converge. This function works in the case of one dimension or multiple dimensions since you'll use it again in problem 2.

Take a few minutes to study this function to make sure you understand how it works and what it is doing. Write a few sentences explaining how this function works.

```
gradient_desc <- function(starting, f, gradient, max = F, maxit = 200,
                          step_size = 1, line_parameter = 0.5, tol = 1e-10) {
  if(max == T) {
    obj_func <- function(x) {
      -f(x)
    }
    func_deriv <- function(x) {
      -gradient(x)
    }
  } else {
    obj_func <- f
    func_deriv <- gradient
  }
  xk <- starting
  for(k in 1:maxit) {
```

```
    gamma <- step_size
    line_beta <- line_parameter
    # Below is the backtracking line search
    while(obj_func(xk - gamma * func_deriv(xk)) >
          obj_func(xk) - gamma/4 * sum(func_deriv(xk)^2)) {
      gamma <- line_beta * gamma
    }
    # Once we have a good step size, continue
    xnew <- xk - gamma * func_deriv(xk)
    if(norm(xk - xnew, type = "2") < tol) {
      break
    }
    xk <- xnew
    }
  return(list(x = xk, function_val = f(xk), iters = k))
}
```

Part c (5 points)

Use the gradient descent function to find the global **min** of $f(x) = \ln(1+x^2)(1+x)e^{-x^2}$. Try using several starting points. Keep track of and report the number of iterations needed to converge at the different starting points.

Part d (4 points)

Use the gradient descent function to find the global **max** of $f(x) = \ln(1+x^2)(1+x)e^{-x^2}$. Again, try several starting points. Keep track of and report the number of iterations needed to converge at the different starting points.

Part e (8 points)

Use the fact that

$$f''(x) = \frac{2e^{-x^2}}{(1+x^2)^2}\left((2x^3+2x^2-3x-1)(1+x^2)^2\ln(1+x^2) - 4x^5 - 4x^4 - 3x^3 - 5x^2 + 3x + 1\right)$$

to implement Newton's method to find the global max **and** the global min. Use the same starting points you did in parts c and d. Keep track of and report the number of iterations needed to converge at the different starting points. And yes, that second derivative is very messy! This is one reason why Newton's method is less popular. You don't have to write a function for this part to implement Newton's method, but you can if you'd like.

Part f (3 points)

Compare the number of iterations needed for convergence for gradient descent and for Newton's method.

Part g (4 points)

Use the `optimize()` function in **R** to find the global max and min for the function f.

Problem 2 (26 points)

Suppose $x = (x_1, \dots, x_n)^T$ are independent and identically distributed random variables with probability density function (pdf) given by

$$f(x_i|\theta) = \theta(1 - x_i)^{\theta-1}; \quad 0 \leq x_i < 1, \ 0 < \theta < \infty, \ i = 1, \dots, n$$

Part a (4 points)

Using ggplot, plot the probability density function (pdf) for an individual x_i given $\theta = 0.5$ and then again for $\theta = 5$.

Part b (5 points)

Give the likelihood $L(\theta|x)$ and log-likelihood $\ell(\theta|x)$ functions in this situation.

Part c (5 points)

Find the Maximum Likelihood Estimator (MLE) $\widehat{\theta}$ for θ.

Part d (3 points)

Show that your estimator is in fact a maximum. That is, check the second derivative of the log-likelihood.

Part e (2 points)

Find the MLE if the data values are given below. Note, the actual θ value used to generate these data was $\theta = 7.3$.

```
x <- c(0.0194, 0.0053, 0.2488, 0.0456, 0.2059, 0.0992, 0.1168, 0.3705,
       0.2129, 0.018, 0.0464, 0.1401, 0.0759, 0.1588, 0.0334, 0.2931,
       0.0662, 0.2292, 0.1581, 0.3462, 0.035, 0.1086, 0.0793, 0.2095,
       0.1419, 0.1835, 0.1107, 0.0764, 0.1331, 0.042, 0.0911, 0.1608)
```

Part f (7 points)

To get an idea of how variable the maximum likelihood estimator is in this case, let's generate many samples of size 32 from this distribution (which is a Beta distribution with parameters 1 and θ, but you don't really need to know that). We can do this by using `rbeta(32, 1, 7.3)`.

Using the code below, generate at least 10,000 samples this way, compute the MLE for each, and then plot a histogram of the values using ggplot. Plot a vertical line at the true θ value of 7.3 (you can use `geom_vline(xintercept = 7.3)` for this). On the plot somewhere, indicate what the standard deviation in those estimates is. Of course, this code is incomplete, so you'll have to complete it. Once you do, make sure to change the `eval` option to `true`.

Once you've completed the code for the samples of size 32, repeat this but instead of taking samples of size 32, take samples of size 100. How does this change the histogram, where the distribution is centered, and the standard deviation of the estimates, if at all?

```
set.seed(2026)
theta_hat <- rep(0, 10000) # Initialize the vector
for(i in 1:10000) {
  x_sim <- rbeta(32, 1, 7.3)
  theta_hat[i] <- # Put the code for the MLE here
}
# Fill in code below for the plot
```

Problem 3 (31 points)

We implemented optimization to maximize the likelihood for a Poisson regression problem in the notes to estimate the β (intercept and slope) values. Let's do something similar for find the β vector by maximizing the likelihood in logistic regression.

In logistic regression, we attempt to predict the probability of a success for a given case. We can use the Bernoulli random variable for this. For a Bernoulli random variable, $P(Y = y_i) = p_i^{y_i}(1-p_i)^{1-y_i}$ for $y_i = 0, 1$ where p_i is the probability of a success. In our case, for logistic regression, we have $\text{logit}(p_i) = \ln\left(\dfrac{p_i}{1-p_i}\right) = \mathbf{X}_i\beta$. That means $p_i = \dfrac{\exp(\mathbf{X}_i\beta)}{1+\exp(\mathbf{X}_i\beta)}$.

The likelihood function, $L(\beta|\mathbf{X}, y)$, for the logistic regression for a sample of size n is

$$\begin{aligned} L(\beta|\mathbf{X}, y) &= \prod_{i=1}^{n} p_i^{y_i}(1-p_i)^{1-y_i} \\ &= \prod_{i=1}^{n} \left[\frac{\exp(\mathbf{X}_i\beta)}{1+\exp(\mathbf{X}_i\beta)}\right]^{y_i} \left(1 - \left[\frac{\exp(\mathbf{X}_i\beta)}{1+\exp(\mathbf{X}_i\beta)}\right]\right)^{1-y_i} \end{aligned}$$

Part a (7 points)

Using the above likelihood, show that the log likelihood function is

$$\ell(\beta|\mathbf{X}, y) = \sum_{i=1}^{n} y_i \ln\left(\frac{\exp(\mathbf{X}_i\beta)}{1+\exp(\mathbf{X}_i\beta)}\right) + \sum_{i=1}^{n}(1-y_i)\ln\left(\frac{1}{1+\exp(\mathbf{X}_i\beta)}\right)$$

and then show it can be equivalently written

$$\ell(\beta|\mathbf{X}, y) = \sum_{i=1}^{n} y_i(\mathbf{X}_i\beta) - \sum_{i=1}^{n} \ln\Big(1+\exp(\mathbf{X}_i\beta)\Big)$$

using properties of logarithms.

Part b (6 points)

Find the gradient of the log likelihood if we have three β's. That is, if $\mathbf{X}_i\beta = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2}$.

Part c (4 points)

In Homework 3, Problem 2, part a, you read in the `adult` dataset (from the UC Irvine database). This is the one that we used to predict whether a person makes over \$50K a year based on some other variables. The data came from the Census Bureau in 1994 and can be found in the Data folder in my Math3190_S26 GitHub repo. More info on the dataset can be found in the "adult.names" file.

Read in the dataset and put column names like you did in HW 3. Then fit a logistic regression model using the `glm()` function that predicts salary from age/10 and sex. Note, we should divide the age by 10 in our model because this makes the gradient descent more stable (for whatever reason). We can divide by 10 in the `glm()` function by wrapping it in the `I()` function. Like this: `glm(salary ~ I(age/10) + sex, data = adult, family = binomial)`.

Part d (4 points)

Define `y`, `x1`, and `x2`. `y` should be a vector of 0's and 1's with a 1 indicating that the person had a salary above \$50,000. `x1` should be a vector that contains all of the age values (divided by 10) and then `x2` should be an indicator variable that indicates if the person is male or not. Taking columns from the matrix obtained using the `model.matrix()` function applied to your logistic regression model you fit in part c may be useful here.

Part e (7 points)

Use the gradient descent function from problem 1 to find estimates for β_0, β_1, and β_2 for predicting salary from age and sex. Enter the following in your function:

- Use the vector `c(0, 0, 0)` as your starting point.
- Instead of starting the step size, γ_k, at 1, start it at 0.1 to save some time with the line search.
- Make sure the maximum number of iterations is at least 200.

You'll know you did this right if the optimized values you end up with match (or are very close to) the estimates from your logistic regression model you fit in part c. This may take a minute or two to run, so you may want to cache this code chunk.

Part f (3 points)

Use the `optim()` function in **R** to find the estimate for the β's here. This should be much faster since this function is much better optimized (no pun intended). The results here should also be very close to the numbers we obtained for the slopes in the model we fit in part c, but may not match exactly due to different stopping criteria.